Doramagic 产品定义手册

> **版本**：v3.0 完整版
> **日期**：2026-03-11
> **状态**：✅ 已确认
> **作者**：G大脑袋（产品架构师）
> **审核**：Tangsir（创始人）

一、工作方法

> **讨论原则**：第一性原理沟通，一次讨论一个问题，追问本质。

本文档是 Doramagic 产品的完整定义，基于 2026-03-11 与 Tangsir 的深度讨论整理。所有核心概念均经过第一性原理拷问验证。

二、核心理念

2.1 一句话定义

> **Doramagic = 超级 Copy Cat —— 站在巨人肩膀上快速抄作业**

2.2 核心理念

维度	说明
不是	帮用户从 0 开发一个功能
而是	从开源项目中找到成熟的解决方案，复制 + 调整
本质	用 AI 大模型替代人肉抄作业，自动完成知识提取和能力匹配

2.3 初心追溯

在与 Tangsir 的讨论中，我们进行了深刻的第一性原理拷问：

> 「没有 Doramagic，用户会死吗？」
> 「有了 Claude，用户为什么还需要你？」

Tangsir 的回答：

- 用 SupPower 时觉得好用，但不适合自己
- 以前只能忍受，AI 时代可以调整
- 长期记忆技术是这种范式的典型

这揭示了核心洞察：**不是「好」，而是「适合」**。

三、范式转变

3.1 核心洞察：「好」vs「适合」

之前（工具时代）	现在（AI 时代）
用户适应工具	工具适应用户
忍受不喜欢的功能	AI 动态调整
学习成本：高	使用成本：零

3.2 范式对比

旧范式	新范式（Doramagic）
用户适应工具	**工具适应用户**
学习技能才能用	**直接给成品**
忍受不喜欢的功能	**AI 动态调整到适合**

旧范式	新范式（Doramagic）
搜索 → 学习 → 配置 → 使用	**需求 → 成品**

3.3 竞品差异化

产品	定位	比喻
Cursor/Copilot	建筑师	从 0 开始构建
Doramagic	抄作业专家	复制 + 调整
SupPower	工具集合	准备好一堆工具，用户自己选

四、产品定位

4.1 定位声明

Doramagic 是 AI 时代的「抄作业专家」—— 让用户快速获得能用的成品，而不是教用户开发技能。

4.2 目标用户

用户类型	特征	需求
Vibe Coding 新手	用 Cursor/Claude Code 写代码	快速解决「配置问题/版本 bug/踩坑」
效率至上者	不想学只想用	直接要成品，不要学习成本
跨领域探索者	多个领域浅尝辄止	需要快速进入新领域，不深入学习

4.3 核心价值

价值点	说明
零学习成本	不用说「教我开发」，直接说「我要」
个性化适配	记住用户的「适合」，动态调整
踩坑经验	整合 GitHub Issues/Discussions 的真实踩坑记录
快速动起来	不是给你渔具，是直接把鱼给你

五、三个核心挑战

5.1 挑战清单

挑战	优先级	说明
噪音过滤	⭐⭐⭐ 最高	准确识别真正的知识和经验，是产品成立的前提
关联推理	⭐⭐ 很高	用户表达可能模糊/缺失，如何准确映射到开源项目
时效性	⭐ 次要	知识是沉淀的，不需要实时性

5.2 噪音过滤策略

> **本质问题**：开源社区充满噪音——重复问题、无效讨论、过时答案

解决思路：

1. **来源分级**：官方文档 > 官方 Issue > 社区讨论 > 第三方博客
2. **质量评分**：基于 upvotes、是否被官方采纳、解决方案验证
3. **LLM 过滤**：用 LLM 判断内容是否为「有效知识」

5.3 关联推理策略

> **本质问题**：用户说「我想管 Trello」→ 映射到哪个开源项目？哪个能力？

解决思路：

1. **置信度机制**：70% 阈值，高于直接执行，低于追问
2. **选择题优先**：不让用户自己想选项，而是给选项
3. **上下文匹配**：结合用户已有的 Skills、记忆、模型做个性化推荐

六、触发词设计

6.1 触发词

触发词	语言	说明
Dora	通用	简洁首选
哆啦A梦	中文	亲切感
ドラえもん	日文	语言变体
Doraemon	英文	语言变体

6.2 风格定义：Doraemon 风格

维度	定义
语气	俏皮、友好、有帮助
动作感	从「口袋」掏工具的感觉
交互	选择题优先，不让用户组织语言
透明度	用户知道在做什么，实时进度可见

6.3 示例交互

```
用户: Dora 我想管 Trello

Doramagic: 了解啦！让我从口袋里找找看～🎁

你是想用 Trello 做什么呢?

A) 日常管理任务 – 创建卡片、移到不同列表、标记完成
B) 同步进度 – 查看各 Board 卡片、追踪任务状态
C) 团队协作 – 留评论、@人、分享卡片
```

七、用户交互设计

7.1 自适应交互深度

> **核心洞察**：不是所有用户都需要深度交互

用户类型	需求清晰度	需要的交互
能手型	说得很清楚	0 轮，直接进入选择
普通型	大概知道	1-2 轮选择题
新手型	说不清楚	3-5 轮追问

7.2 置信度机制

参数	值	说明
阈值	70%	行业标准（Dialogflow、LivePerson 均采用）
高于阈值	直接呈现	进入能力积木阶段
低于阈值	选择题追问	最多 3 轮
达到上限	强制进入	提供保守方案

7.3 选择题设计原则

原则	实现
选择题优先	不让用户自己组织语言
最多 3 轮	超过后主动提供保守方案
提前终止	用户一旦选了就立刻推进
复述确认	「你是说 xxx 对吧？」形成闭环

7.4 完整交互流程

```
用户输入
    ↓
【第一阶段：自适应交互】
    ↓ 置信度判断
【第二阶段：意图理解 + 资源检索】← 子代理并行
    ↓
【第三阶段：能力积木呈现】
    ↓
【第四阶段：用户选择 + 配置】
    ↓
【第五阶段：Skill 生成 + 交付】
```

八、能力积木体系

8.1 四种积木类型

```
┌──────────────────────────────────────────────────────────┐
│                        能力积木体系                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐    │
│  │   触发积木   │  │   能力积木   │  │   资源积木   │    │
```

```
│  │  Trigger    │  │   Action     │  │  Resource   │       │
│  ├──────────────┤  ├──────────────┤  ├──────────────┤       │
│  │ 触发关键词   │  │ 操作命令     │  │ API 端点    │       │
│  │ 使用场景    │  │ 输入/输出    │  │ 工具推荐    │       │
│  │ 用户意图    │  │ 配置要求     │  │ 信息源      │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  ┌──────────────┐                                          │
│  │   依赖积木    │                                          │
│  │  Dependency │                                          │
│  ├──────────────┤                                          │
│  │ 环境变量     │                                          │
│  │ 安装要求     │                                          │
│  │ 权限需求     │                                          │
│  └──────────────┘                                          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

8.2 积木属性定义

```
block:
 # 基础信息
 id: "trello-list-boards"
 name: "查看所有 Board"
 type: "action"  # trigger | action | resource | dependency

 # 来源追踪
 source:
   project: "trello"
   repo: "github.com/trello/trello"
   extracted_at: "2026-03-11"
   extraction_type: "code|issue|doc|manual"

 # 能力定义
 capability:
   verb: "list"
   noun: "boards"
   description: "查看用户所有的 Board 列表"

 # 技术细节
```

```
  tech:
    type: "rest_api"
    endpoint: "GET /1/members/me/boards"
    params: [...]
    output: {...}

  # 经验沉淀
  experience:
    common_issues: [...]
    best_practices: [...]

  # 资源推荐
  resources:
    apis: [...]
    tools: [...]
    alternatives: [...]

  # 依赖信息
  dependencies:
    env: [...]
```

8.3 资源来源

来源	类型	说明
GitHub 开源项目	代码 + Issues + Discussions	核心知识来源
OpenClaw Skills	已有能力	复用现有能力
用户上下文	记忆 + 偏好 + 模型	个性化匹配

8.4 四层提取体系

层次	内容	来源
代码层	设计意图、架构决策	代码分析
社区层	踩坑经验、workaround	Issues/Discussions

层次	内容	来源
资源层	API、工具链、信息源	项目生态
上下文层	记忆、偏好、模型	用户实时数据

九、「四足够」设计原则

9.1 足够专业：分析用户上下文

用户资源	积木呈现策略
有 Claude 模型	优先推荐需要深度理解的能力
有 MiniMax 模型	优先推荐需要多轮对话的能力
记忆文件有股票	优先推荐 A 股相关能力
已安装 Trello Skill	问「要增强 Trello 能力吗？」
macOS 系统	优先推荐 macOS 原生能力

9.2 足够智能：子代理并行

```
用户: Dora 我想管 Trello
            ↓
┌──────────────────────────────────────────────┐
│            Doramagic 内部处理                  │
├──────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐          │
│  │ 意图理解     │  │ 资源检索     │  ← 并行  │
│  │ 子代理       │  │ 子代理       │          │
│  └──────────────┘  └──────────────┘          │
│          ↓                ↓                   │
│  ┌──────────────┐  ┌──────────────┐          │
│  │ 置信度评估   │  │ 积木匹配     │  ← 并行  │
```

```
│   │ 子代理        │   │ 子代理        │             │
│   └──────────────┘   └──────────────┘             │
└──────────────────────────────────────────────────┘

                ↓
一次性呈现完整结果
```

9.3 足够适用：开箱即用 + 配置引导

```
三层配置状态金字塔：


┌──────────────┐
│  开箱即用     │ ← 默认推荐，0 配置
│  (0 配置)    │
└──────────────┘
       │
       │
┌──────────────┐
│  轻量配置     │ ← 提示建议，1-2 项
│  (1-2 项)    │
└──────────────┘
       │
       │
┌──────────────┐
│  完整配置     │ ← 进阶用户，3+ 项
│  (3+ 项)     │
└──────────────┘
```

9.4 足够透明：实时进度可见

```
┌──────────────────────────────────────────────────────┐
│  🎯 正在分析你的需求...                                 │
│  ▓▓▓
```

十、技术架构

10.1 架构选型历程

阶段	方案	问题
最初	LangChain + Qdrant + BGE + Mem0 + Neo4j	太重，不是 Skill
最终	纯 LLM + OpenClaw Skills + 记忆文件	轻量、纯 Skill

10.2 最终轻量架构

```
用户 → Doramagic Skill
        ↓
┌──────────────────────────────────────┐
│           纯 LLM 能力                 │
│  • 意图理解（Prompt）                 │
│  • Skill 规划（LLM）                  │
│  • 上下文读取（记忆文件）             │
│  • 结果组装生成                       │
└──────────────────────────────────────┘
        ↓
  Skill 调用层
  ┌──────────────┐
  │ Skills       │ ← OpenClaw 已有
  │ LLM          │ ← OpenClaw 已配置
  │ 记忆         │ ← 记忆文件
  └──────────────┘
```

10.3 零外部依赖

资源	来源
LLM	OpenClaw 已配置
Skills	OpenClaw 已有

资源	来源
知识存储	记忆文件
灵魂提取	已有 soul-extractor Skill

10.4 关键技术组件

组件	功能	技术方案
意图识别	理解用户想要什么	LLM + Prompt Engineering
置信度评分	判断需求是否清晰	四维度加权模型
选择题生成	生成合理的选项	LLM 生成 + 知识库检索
对话状态	管理多轮对话状态	内存状态机
灵魂提取	从开源项目提取知识	soul-extractor Skill（已存在）

10.5 置信度评分维度

维度	权重	说明
实体完整度	35%	用户输入中关键实体是否完整
意图匹配度	25%	LLM 对意图判断的确定性
模糊词检测	20%	检测「随便」「都行」等模糊词
上下文连贯性	20%	与历史对话的连贯程度

十一、长期记忆系统

11.1 核心方向

记住用户的「适合」—— 动态调整，而非固定选项。

11.2 记忆维度

维度	说明	示例
喜欢什么方式	交互风格偏好	简洁 vs 详细
跳过什么建议	不关注的选项	跳过配置建议
选择什么配置	最终决策模式	总是选高级选项
关注什么信息	注意力焦点	关注性能而非易用性

11.3 记忆工作原理

用户 A 示例：

1. 说「帮我做个 Trello 管理」
2. Doramagic 给出 4 个选项：A/B/C/D
3. 用户 A 选了「简洁模式」
4. → 长期记忆：**A 选择了简洁模式，不想要复杂配置**
5. 下次用户 A 说「帮我做个日历管理」
6. → Doramagic 直接默认给简洁模式，跳过复杂选项

用户 B 示例：

1. 每次都选 D（高级自定义）
2. → 长期记忆：**B 喜欢深度定制**
3. 下次给更多高级选项

11.4 技术实现（待细化）

- **存储**：OpenClaw 记忆文件
- **更新**：每次用户决策后更新

- **检索**：查询时加载用户偏好
- **融合**：在选择题生成时融入偏好

十二、用户场景

场景 1：告警规则不生效

方案	回答
通用 LLM	告诉你「在 alertmanager.yml 里加 group_by 参数」
Doramagic	代码层面：为什么要这么配 + 社区层面：GitHub Issue 里的 bug report、踩坑经验、最新 workaround

场景 2：Vibe Coding 新手

- 用 Cursor/Claude Code 写代码
- 遇到问题和实际跑起来的结果不一样
- 需要快速定位是「配置问题」还是「版本 bug」还是「踩过的坑」

场景 3：跨领域探索

- 用户想做「每日任务提醒」
- 传统方式：搜索 → 学习 → 配置 → 使用
- Doramagic 方式：需求 → 成品（直接给一个可用的 Skill）

十三、与现有系统的关系

13.1 调用已有能力

能力	关系	说明
soul-extractor	调用	已有 Skill，用于深度提取开源项目灵魂

能力	关系	说明
OpenClaw Skills	复用	已有 Skills 作为积木来源
LLM	调用	已配置的 LLM（Gemini/Ollama）
记忆系统	共用	使用 OpenClaw 的记忆文件

13.2 差异化定位

Skill	定位	特点
soul-extractor	深度提取	时间长、质量高
Doramagic	快速规划	时间短、智能组合

十四、商业模式（待讨论）

> ⚠️ 本节为开放问题，待与 Tangsir 讨论确定

可能的商业模式方向：

1. **免费增值**：基础功能免费，高级能力付费
2. **按量计费**：按提取次数/知识卡片数量计费
3. **企业版**：私有部署、定制化服务
4. **生态分成**：与开源项目合作，获得知识授权

十五、待讨论问题

- [ ] 长期记忆的技术实现方案
- [ ] Doramagic 具体的产品形态（独立 Skill vs 功能模块）
- [ ] 商业模式
- [ ] 定价策略

- [ ] 冷启动策略（如何获取首批积木）

十六、相关文档

文档	位置	状态
soul-extractor Skill	~/.agents/skills/soul-extractor/	✅ 已存在
技术方案研究报告	~/Documents/openclaw/Doramagic/research/	✅ 已生成
能力积木设计文档	~/Documents/openclaw/Doramagic/research/	✅ 已生成
自适应交互设计	~/Documents/openclaw/Doramagic/research/	✅ 已生成
技术架构方案	~/Documents/openclaw/Doramagic/research/	✅ 已生成
2026 年最先进技术方案	~/Documents/openclaw/Doramagic/research/	✅ 已生成

十七、版本历史

版本	日期	说明
v1.0	2026-03-11	产品手册版本，双半魂模型
v2.0	2026-03-11	讨论中版本，超级 Copy Cat 定位
v3.0	2026-03-11	本版本，基于完整讨论的第一性原理重构

本文档基于 2026-03-11 与 Tangsir 的深度讨论整理

核心理念：不是「帮你做」，而是「帮抄作业」

范式转变：工具适应用户，而不是用户适应工具